- Transacciones
  - ✅ Ejercicio 1
  - ✅ Ejercicio 2
  - ✅ Ejercicio 3
  - ✅ Ejercicio 4
  - ✅ Ejercicio 5
  - ✅ Ejercicio 6
  - ✅ Ejercicio 7
  - ✅ Ejercicio 8
  - ✅ Ejercicio 9
  - ✅ Ejercicio 10
  - ✅ Ejercicio 11

Transacciones

Realiza los siguientes ejercicios sobre la base de datos **empresa**

✅ Ejercicio 1

Incrementa un **10 %** el salario del empleado **7369** y un **5 %** el del empleado **7499** dentro de una misma transacción. Comprueba los nuevos salarios y **confirma** los cambios. Intenta después ejecutar `ROLLBACK` y comprueba si se recuperan los salarios anteriores.

💡 ***Solución***

```
START TRANSACTION;

UPDATE EMP SET SALARI = SALARI * 1.10 WHERE EMP_NO = 7369;
UPDATE EMP SET SALARI = SALARI * 1.05 WHERE EMP_NO = 7499;

SELECT EMP_NO, COGNOM, SALARI
FROM EMP
WHERE EMP_NO IN (7369, 7499);

COMMIT;

ROLLBACK;

SELECT EMP_NO, COGNOM, SALARI
FROM EMP
WHERE EMP_NO IN (7369, 7499);
```

> Los salarios permanecen modificados porque `COMMIT` confirma definitivamente los cambios. El `ROLLBACK` posterior no puede deshacerlos.

✅ Ejercicio 2

Aumenta temporalmente el límite de crédito del cliente **100** en **1000** unidades y el del cliente **109** en **2000** unidades. Comprueba los nuevos valores y **cancela** la transacción. Verifica que ambos límites recuperan su valor inicial.

💡 *Solución*

```
SELECT CLIENT_COD, NOM, LIMIT_CREDIT
FROM CLIENT
WHERE CLIENT_COD IN (100, 109);

START TRANSACTION;

UPDATE CLIENT SET LIMIT_CREDIT = LIMIT_CREDIT + 1000
WHERE CLIENT_COD = 100;

UPDATE CLIENT SET LIMIT_CREDIT = LIMIT_CREDIT + 2000
WHERE CLIENT_COD = 109;

SELECT CLIENT_COD, NOM, LIMIT_CREDIT
FROM CLIENT
WHERE CLIENT_COD IN (100, 109);

ROLLBACK;

SELECT CLIENT_COD, NOM, LIMIT_CREDIT
FROM CLIENT
WHERE CLIENT_COD IN (100, 109);
```

> `ROLLBACK` deshace las dos modificaciones de la transacción, aunque se hayan realizado mediante instrucciones distintas.

✅ Ejercicio 3

Inserta los departamentos **50 (INFORMATICA, VALENCIA)** y **60 (MARKETING, ALICANTE)** en una única transacción. Comprueba que existen y deshaz la operación. Repite después las inserciones en una nueva transacción, pero esta vez **confírmala**.

💡 *Solución*

```sql
START TRANSACTION;

INSERT INTO DEPT (DEPT_NO, DNOM, LOC)
VALUES (50, 'INFORMATICA', 'VALENCIA');

INSERT INTO DEPT (DEPT_NO, DNOM, LOC)
VALUES (60, 'MARKETING', 'ALICANTE');

SELECT * FROM DEPT WHERE DEPT_NO IN (50, 60);

ROLLBACK;

SELECT * FROM DEPT WHERE DEPT_NO IN (50, 60);

START TRANSACTION;

INSERT INTO DEPT (DEPT_NO, DNOM, LOC)
VALUES (50, 'INFORMATICA', 'VALENCIA');

INSERT INTO DEPT (DEPT_NO, DNOM, LOC)
VALUES (60, 'MARKETING', 'ALICANTE');

COMMIT;

SELECT * FROM DEPT WHERE DEPT_NO IN (50, 60);
```

> Tras el primer `ROLLBACK` no queda ninguno de los dos departamentos. Tras `COMMIT`, ambos permanecen guardados.

✅ Ejercicio 4

Traslada al empleado **7369** al departamento **30** y aumenta un **10 %** el salario de todos los empleados de ese departamento, incluido el trasladado. Comprueba los resultados y **deshaz todos los cambios**. ¿Por qué conviene realizar las dos operaciones en una misma transacción?

💡 ***Solución***

```
START TRANSACTION;

UPDATE EMP SET DEPT_NO = 30 WHERE EMP_NO = 7369;

UPDATE EMP SET SALARI = SALARI * 1.10 WHERE DEPT_NO = 30;

SELECT EMP_NO, COGNOM, DEPT_NO, SALARI
FROM EMP
WHERE DEPT_NO = 30 OR EMP_NO = 7369;

ROLLBACK;

SELECT EMP_NO, COGNOM, DEPT_NO, SALARI
FROM EMP
WHERE DEPT_NO = 30 OR EMP_NO = 7369;
```

Las operaciones están relacionadas: si se cancela el traslado, también debe cancelarse la actualización salarial asociada. La transacción permite deshacerlas conjuntamente.

✅ Ejercicio 5

Inicia una transacción e inserta un empleado con código **9001** en el departamento **20**. A continuación, intenta insertar otro empleado con código **9002** en el departamento **99**, que no existe. Comprueba qué ocurre con la segunda inserción y si la primera sigue pendiente dentro de la transacción. **Cancela** finalmente la operación completa.

💡 *Solución*

```
START TRANSACTION;

INSERT INTO EMP (EMP_NO, COGNOM, OFICI, SALARI, DEPT_NO)
VALUES (9001, 'PEREZ', 'ANALISTA', 200000, 20);

-- Esta instrucción falla porque no existe el departamento 99.
INSERT INTO EMP (EMP_NO, COGNOM, OFICI, SALARI, DEPT_NO)
VALUES (9002, 'LOPEZ', 'ANALISTA', 180000, 99);

SELECT EMP_NO, COGNOM, DEPT_NO
FROM EMP
WHERE EMP_NO IN (9001, 9002);

ROLLBACK;

SELECT EMP_NO, COGNOM, DEPT_NO
FROM EMP
WHERE EMP_NO IN (9001, 9002);
```

La segunda inserción incumple la clave ajena `DEPT_NO` y produce un error. En MariaDB/InnoDB, ese error **no deshace automáticamente toda la transacción**: el empleado 9001 sigue insertado provisionalmente hasta que se ejecuta `ROLLBACK`. Después no permanece ninguno de los dos empleados.

✅ Ejercicio 6

La empresa decide aumentar inicialmente un **5 %** los salarios del departamento **20**, pero antes de confirmar la operación cambia de criterio y establece un incremento del **3 %**. Deshaz la primera modificación y aplica la segunda en una **nueva transacción**, confirmando únicamente el incremento definitivo.

💡 *Solución*

```sql
SELECT EMP_NO, COGNOM, SALARI
FROM EMP
WHERE DEPT_NO = 20;

START TRANSACTION;

UPDATE EMP SET SALARI = SALARI * 1.05
WHERE DEPT_NO = 20;

ROLLBACK;

START TRANSACTION;

UPDATE EMP SET SALARI = SALARI * 1.03
WHERE DEPT_NO = 20;

COMMIT;

SELECT EMP_NO, COGNOM, SALARI
FROM EMP
WHERE DEPT_NO = 20;
```

El primer incremento se cancela por completo. El segundo se calcula sobre los salarios originales y queda confirmado. `ROLLBACK` finaliza la primera transacción; por eso es necesario iniciar otra.

✅ Ejercicio 7

En la comanda **610**, modifica la cantidad del detalle **3** a **5 unidades**. Actualiza también el importe de esa línea multiplicando la cantidad por el precio de venta y recalcula el total de la comanda como la

suma de los importes de sus detalles. Realiza las tres modificaciones dentro de una misma transacción y **confírmalas** después de comprobar los resultados.

💡 ***Solución***

```
START TRANSACTION;

UPDATE DETALL
SET QUANTITAT = 5
WHERE COM_NUM = 610 AND DETALL_NUM = 3;

UPDATE DETALL
SET IMPORT = QUANTITAT * PREU_VENDA
WHERE COM_NUM = 610 AND DETALL_NUM = 3;

UPDATE COMANDA
SET TOTAL = (
    SELECT SUM(IMPORT)
    FROM DETALL
    WHERE DETALL.COM_NUM = COMANDA.COM_NUM
)
WHERE COM_NUM = 610;

SELECT COM_NUM, DETALL_NUM, QUANTITAT, PREU_VENDA, IMPORT
FROM DETALL
WHERE COM_NUM = 610;

SELECT COM_NUM, TOTAL
FROM COMANDA
WHERE COM_NUM = 610;

COMMIT;
```

La cantidad, el importe del detalle y el total de la comanda deben ser coherentes entre sí. Agrupar las operaciones en una transacción evita confirmar una actualización incompleta.

✅ Ejercicio 8

Alta de un pedido. Registra un pedido nuevo con número **700**, correspondiente al cliente **100**, con fecha **2026-10-28** y tipo **A**. El pedido contiene dos líneas: **2 unidades** del producto **100890** a **58** unidades monetarias cada una y **3 unidades** del producto **100870** a **2,80** unidades monetarias cada una. Inserta la cabecera en `COMANDA` y las líneas en `DETALL` dentro de una misma transacción. Calcula el total del pedido, comprueba las inserciones y **confirma** los cambios. ¿or qué conviene agrupar las tres inserciones en una transacción?.

💡 ***Solución***

```sql
START TRANSACTION;

INSERT INTO COMANDA (COM_NUM, COM_DATA, COM_TIPUS, CLIENT_COD, TOTAL)
VALUES (700, '2026-10-28', 'A', 100, 124.40);

INSERT INTO DETALL (COM_NUM, DETALL_NUM, PROD_NUM, PREU_VENDA, QUANTITAT, IMPORT)
VALUES (700, 1, 100890, 58.00, 2, 116.00);

INSERT INTO DETALL (COM_NUM, DETALL_NUM, PROD_NUM, PREU_VENDA, QUANTITAT, IMPORT)
VALUES (700, 2, 100870, 2.80, 3, 8.40);

SELECT * FROM COMANDA WHERE COM_NUM = 700;
SELECT * FROM DETALL WHERE COM_NUM = 700;

COMMIT;
```

El total es **124,40** (116 + 8,40). La transacción permite confirmar el pedido completo, con su cabecera y sus líneas, o deshacer todas las inserciones si surge algún problema antes de `COMMIT`. El código 700 debe estar libre antes de ejecutar el ejercicio.

✅ Ejercicio 9

Modificación de un pedido. En el pedido **611**, modifica la cantidad del detalle **1** de **1** a **4 unidades**. Recalcula su importe y actualiza el total de `COMANDA` para que coincida con la suma de los importes de `DETALL`. Comprueba los valores obtenidos.
La empresa decide finalmente no aplicar el cambio: **deshaz toda la transacción** y verifica que se recuperan los valores originales.

💡 ***Solución***

```sql
SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;
SELECT COM_NUM, DETALL_NUM, QUANTITAT, PREU_VENDA, IMPORT
FROM DETALL WHERE COM_NUM = 611;

START TRANSACTION;

UPDATE DETALL
SET QUANTITAT = 4,
    IMPORT = 4 * PREU_VENDA
WHERE COM_NUM = 611 AND DETALL_NUM = 1;

UPDATE COMANDA
SET TOTAL = (
    SELECT SUM(IMPORT)
    FROM DETALL
    WHERE DETALL.COM_NUM = COMANDA.COM_NUM
)
WHERE COM_NUM = 611;

SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;
SELECT COM_NUM, DETALL_NUM, QUANTITAT, PREU_VENDA, IMPORT
FROM DETALL WHERE COM_NUM = 611;

ROLLBACK;

SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;
SELECT COM_NUM, DETALL_NUM, QUANTITAT, PREU_VENDA, IMPORT
FROM DETALL WHERE COM_NUM = 611;
```

Durante la transacción, el importe del detalle y el total del pedido pasan de **45** a **180**. Tras `ROLLBACK`, ambos recuperan el valor **45** y la cantidad vuelve a **1**. Agrupa las modificaciones evita que el total del pedido y sus detalles queden descoordinados.

✅ Ejercicio 10

Cancelación de un pedido. Cancela el pedido **611** eliminando sus detalles y después su cabecera. Antes de hacerlo, intenta borrar directamente el registro de `COMANDA` y observad qué ocurre. A continuación, inicia una transacción, elimina los registros en el orden correcto y comprueba que han desaparecido. **Deshaz** la transacción para conservar el pedido original. ¿Qué restricción de integridad impide eliminar primero la cabecera?.

💡 ***Solución***

```sql
-- Esta operación falla: hay detalles que hacen referencia al pedido.
DELETE FROM COMANDA WHERE COM_NUM = 611;

START TRANSACTION;

DELETE FROM DETALL WHERE COM_NUM = 611;
DELETE FROM COMANDA WHERE COM_NUM = 611;

SELECT * FROM DETALL WHERE COM_NUM = 611;
SELECT * FROM COMANDA WHERE COM_NUM = 611;

ROLLBACK;

SELECT * FROM COMANDA WHERE COM_NUM = 611;
SELECT * FROM DETALL WHERE COM_NUM = 611;
```

La clave ajena `DETALL.COM_NUM` impide eliminar una cabecera que todavía tiene líneas asociadas. Primero se borran los detalles y después la cabecera. Como se ejecuta `ROLLBACK`, se recuperan ambos registros. Si se quisiera cancelar definitivamente el pedido, se utilizaría `COMMIT` en su lugar.

✅ Ejercicio 11

Confirmación y cancelación de operaciones. Sobre el pedido **611**, aumenta su total en **10** unidades monetarias dentro de una transacción y confírmalo con `COMMIT`. Intenta después deshacer el cambio con `ROLLBACK` y comprueba el resultado. A continuación, inicia una nueva transacción, aumenta de nuevo el total en **10**, pero esta vez ejecuta `ROLLBACK` **antes** de confirmar. ¿Cuál es la diferencia entre ambos casos?.

💡 *Solución*

```sql
SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;

START TRANSACTION;
UPDATE COMANDA SET TOTAL = TOTAL + 10 WHERE COM_NUM = 611;
COMMIT;

ROLLBACK;
SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;

START TRANSACTION;
UPDATE COMANDA SET TOTAL = TOTAL + 10 WHERE COM_NUM = 611;
ROLLBACK;

SELECT COM_NUM, TOTAL FROM COMANDA WHERE COM_NUM = 611;
```

El primer incremento queda guardado porque se confirma con `COMMIT`; el `ROLLBACK` posterior no lo revierte. El segundo incremento sí se deshace porque aún no se había confirmado. **Nota:** este ejercicio modifica solo el total de la cabecera para observar el efecto de `COMMIT` y `ROLLBACK`; en una operación real habría que mantenerlo coherente con los detalles del pedido. Para recuperar el valor inicial tras la práctica, puede ejecutarse `UPDATE COMANDA SET TOTAL = TOTAL - 10 WHERE COM_NUM = 611;`.